Ders 12 Genişletilmiş Notu: Öğrenci Proje Sunumları: k-Boyutlu Ağaçlar, Hash Fonksiyonlarına Saldırılar ve Blokzincir

Ders 12 Genişletilmiş Notu: Öğrenci Proje Sunumları: k-Boyutlu Ağaçlar, Hash Fonksiyonlarına Saldırılar ve Blokzincir

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

Bu ders öğrenci proje sunumlarına ayrılmıştır. Üç sunum yapıldı. Not yalnız sunumların akademik içeriğini ve hocanın akademik yorumlarını aktarır; sunum yapanlar "bir öğrenci" diye anılır. İkinci sunum İngilizce yapıldı ve kaydın sesi büyük ölçüde anlaşılmıyor; o bölüm ağırlıklı olarak ekrandaki slaytlardan kuruldu.

İçindekiler

- 1. k-Boyutlu Ağaçlar: Motivasyon ve Tarihçe
- 2. Literatür Taraması: k-d Ağacının Güncel Kullanımları
- 3. k-d Ağacının Yapısı: Düğümler ve Yapraklar
- 4. Bellek İçi ve Bellek Dışı Erişim Karmaşıklığı
- 5. Aralık Sorgusu ve Disk Düzeni
- 6. Prototip Uygulama: Türkiye İl Merkezleri Üzerinde k-d Ağacı
- 7. Hocanın Yorumları ve Sorular (Birinci Sunum)
- 8. Hash Fonksiyonlarının Saldırılara Karşı Karşılaştırılması
- 9. Blokzincir İçin Algoritmalar ve Hash Fonksiyonları
- Örnek Sorular

1. k-Boyutlu Ağaçlar: Motivasyon ve Tarihçe

Birinci sunumun konusu, öğrencinin kendi önerdiği k-boyutlu ağaçlar (k-d tree) algoritmasıydı. Sunum bir literatür taraması ve bir prototip uygulamadan oluşuyordu. Prototip, taranan çalışmalardan birinin bir kavram ispatı (proof of concept) olarak yeniden gerçeklenmesiydi.

Temel amaç veriye hızlı erişmek ve veriyi sistematik biçimde saklamaktır. Tek boyutlu veri için bunun en bilinen yolu ikili arama ağacıdır: küçük elemanlar ağacın soluna, büyük elemanlar sağına yerleştirilir ve arama bu indeks üzerinden yapılır. Ama gerçek dünyadaki verinin çok azı tek boyutludur ve teknoloji geliştikçe boyut sayısı artmaktadır. k-d ağacı bu soruna çözüm getirmeye çalışır. Adındaki "d", boyut (dimension) anlamındadır; yani "k boyutlu ağaç".

Algoritma köken olarak 1970'lerde J. L. Bentley tarafından önerildi; 1990'da yine Bentley tarafından hesaplamalı geometri konferansında (Symposium on Computational Geometry) sunuldu. Kullanım amaçları üçtür:

- en yakın komşuyu (nearest neighbour) bulmak,
- aranan değerin kendisini bulmak,
- aralık sorgusu (range query) yapmak.

Sunumda, boyut sayısı yüksek olduğunda k-d ağacının aramayı belirgin biçimde hızlandırdığı ve araştırmacılar arasında kabul gördüğü söylendi. Kullanıldığı alanlar: bilgisayar bilimleri,

veritabanı yönetim sistemleri, haritalandırma, meteoroloji ve oşinografi, ağ (network) uygulamaları. Sunumdaki özet cümle şuydu: veri varsa ve çok boyutluysa k-d ağacı ona yatkın bir yapıdır.

2. Literatür Taraması: k-d Ağacının Güncel Kullanımları

Öğrenci altı yayın inceledi. Slayttaki tablo yazar, kurum, başlık, yıl ve katkı sütunlarından oluşuyordu (ekrandan, 00:30:00). Çalışmalar sunumdaki sırayla şöyledir:

1. **"Research on Searching Algorithms for Unstructured Grid Remapping Based on KD Tree" (2020).** Meteoroloji ve oşinografi alanından araştırmacılar. Katkı: dünya düzlemi üzerindeki konumların ve komşu konumların hızlı bulunması için indekslenmesi. Çalışma kaba kuvvet (brute force) arama ile k-d ağacını karşılaştırır: istenen veriye hangisiyle daha hızlı erişilir, verinin boyutu ve hacmi arttığında aradaki fark ne olur? Sonuçta k-d ağacı çok daha performanslı çıkar. Öğrenci bu çalışmanın k-d ağacının önerildiği günden bu yana yeni bir şey sunmadığını, ama yapıyı anlamak için iyi bir çalışma olduğunu söyledi; prototip de bu çalışmanın benzeridir.
2. **"Revisiting kd-tree for Nearest Neighbour Search" (2019).** Bir sanayi araştırma laboratuvarında yapılmış. Katkı: en yakın komşuyu bulmak için klasik k-d ağacından daha performanslı ve ölçeklenebilir bir arama. Sunumda k-d ağacının veritabanı yönetim sistemlerinde sık kullanıldığı ve büyük firmaların hâlâ üzerinde çalıştığı vurgulandı.
3. **"Enhancing the Performance of Flow Classification in SDN-Based Intelligent Vehicular Networks" (2021).** Ağ alanında paket (akış) sınıflandırma problemi. k-d ağacı geometrik uzay da kullanılarak zenginleştirilmiş; paket sınıflandırmayı klasik k-d ağacına göre 24 kata kadar hızlandırdıklarını bildirmişler.
4. **"Efficient and Privacy-Preserving Similarity Range Query Over Encrypted Time Series Data" (2022)** (ekrandan, 00:10:00). Sinyal işleme, konuşma tanıma ve hastalık teşhisi gibi uygulamalarda zaman serisi verisi üzerinde benzerlik sorgusu gerekir: elde bir ses ya da görüntü varsa ona en benzerini bulmak istenir. Araştırmacılar k-d ağacı için verimli ve gizliliği koruyan bir benzerlik aralığı sorgu algoritması tasarlamışlar. Dayanak, k-d ağacının en yakın komşu ve aralık sorgusunu doğal olarak verebilmesidir.
5. **"A Clustering WSN Routing Protocol Based on k-d Tree Algorithm" (2018).** Kablosuz sensör ağlarında (WSN) kümeleme. Kümeleme ne kadar isabetli olursa güç tüketiminde o kadar tasarruf olur ve hizmet kalitesi artar; çünkü kablosuz ağlarda atlama (hop) sayısı önemlidir. Çalışma k-d ağacının burada verimli olduğunu vurgular.
6. **"Kd-tree and adaptive radius (KD-AR Stream) based real-time data stream clustering" (2020).** Akan veri (data stream) üzerinde kümeleme. k-d ağacı üç başka algoritmayla karşılaştırılmış ve en iyi kümeleme performansını vermiş.

Öğrenci bu çalışmaların hepsinin klasik k-d ağacını değil, zenginleştirilmiş sürümlerini kullandığını belirtti; örneğin en yakın komşuyu daha kolay bulmak için uyarlanır yarıçap (adaptive radius) gibi eklentiler. Ortak sonuç: 1970'lerde önerilmesine rağmen k-d ağacı hâlâ geliştirilmekte ve kullanılmaktadır.

3. k-d Ağacının Yapısı: Düğümler ve Yapraklar

Slaytın başlığı "Yöntem: Temeller: Düğümler, Yapraklar ve Karmaşıklık Hesabı" idi (ekrandan, 00:13:30). Slayttaki iki madde:

- Ağaç oluşturulurken düğümler boyutlara göre kırılım (bölme) noktalarını, yapraklar ise veri içeriklerini tutar.
- Verilere erişim bu sayede yalnızca düğümlerde arama tamamlandıktan sonra gerçekleşir.

Şekilde yuvarlaklarla gösterilen iç düğümler (1-6) ve kare kutularla gösterilen yapraklar (a-g) vardır; yanında aynı ağacın düzlemi nasıl dikdörtgen bölgelere ayırdığı çizilmiştir. Yuvarlak kısımlar bir indekstir: veriye gitmek için izlenen yol. Bu yol boyunca hiçbir veriyle karşılaşılmaz; her düğümde yalnız sağa mı sola mı gidileceğine karar verilir. Verinin tamamı en alttaki yapraklarda tutulur. Öğrenci bunu ikili arama ağacıyla karşılaştırdı: ikili arama ağacının bazı gerçeklemelerinde veri doğrudan düğümün içinde tutulur, bu yapıda ise yaprağa gelene kadar veri yoktur.

3.1 Ağacın Oluşturulması

İki boyutlu örnekte $k = 2$'dir. Veri önce birinci boyuta, sonra ikinci boyuta göre sağa ya da sola atılır; bölme sırayla boyutlar arasında döner. Böylece veri indekslenmiş, düğümlenmiş ve yerleştirilmiş olur; yaprağa ulaşıldığında verinin kendisi tutulur. Yöntem üç, dört, beş, genel olarak k boyuta genelleştirilebilir.

Sonraki slaytta iki farklı gösterim vardı (ekrandan, 00:18:30). Soldaki gösterimde düzlem önce dikey çizgilerle r2, r3, r4, r5 bölgelerine, sonra her bölge kesikli yatay çizgilerle dört parçaya (A-P) ayrılır; ağaçta kök r1 dört çocuğa, her çocuk dört yaprağa gider. Sağdaki gösterimde düzlem l1-l9 doğrularıyla bölünür ve noktalar (p1-p10) yapraklarda kalır. Sunumda iki noktaya dikkat çekildi:

- Yapraklara kadar bütün düğümler yuvarlakla gösterilir ve içlerinde veri yoktur; yalnız düzlemin hangi noktalardan bölündüğünü tutarlar. x ve y koordinatları düşünülürse, düzlem kesikli çizgilerle farklı bölgelere kesilir.
- Ağaç her zaman dengeli (balanced) olmaz. Bazı veri yapraklarına kısa yoldan ulaşılırken bazıları daha derinde kalır. Bu, verinin ne kadar iyi dengelendiğine ve bölmeye verinin hangi noktasından başlandığına bağlıdır.

  Ek bilgi (derste anlatılmadı): Klasik k-d ağacında dengeyi sağlamanın yaygın yolu, her seviyede o seviyenin boyutuna göre medyan (ortanca) noktadan bölmektir. Bütün veri baştan biliniyorsa bu yolla yüksekliği $O(\log n)$ olan dengeli bir ağaç kurulabilir. Elemanlar teker teker eklenirse ağaç ikili arama ağacındaki gibi dengesizleşebilir ve en kötü durumda yükseklik n'e yaklaşır.

4. Bellek İçi ve Bellek Dışı Erişim Karmaşıklığı

Öğrenci karmaşıklık hesabında iki ayrı kavramı ayırdı. Ağaç boyunca izlenen yol **bellek içi erişim**dir; bu, önceki derslerde görülen algoritmalardaki "ağaçta ne kadar ilerledim" hesabıdır. Ağacın dışına çıkıldıktan sonraki kısım **bellek dışı erişim**dir: diske kaç kez erişildiği ya da ağ üzerinden kaç paket alınıp gönderildiği gibi işlemler.

Bellek içi erişimler nanosaniye mertebesindeyken bellek dışı erişimler çok daha pahalıdır. Bu yüzden k-d ağacının karmaşıklığı hesaplanırken asıl bakılan bellek dışı erişimdir; "bellek dışı erişim neyi gösteriyorsa karmaşıklık odur" denir, çünkü her işlemde diske gitmek gerekir.

Slayttaki ifadeler (ekrandan, 00:16:25):

- Ağaca eleman eklemenin bellek içi erişim maliyeti, tıpkı ikili arama ağacında olduğu gibi arama yapma maliyetiyle aynıdır: $O(\log n)$. Ağaçta bir yol izlenir ve yolun sonunda veri eklenir.
- Ağaca eleman eklemenin bellek dışı erişim maliyeti ise yaprağa sığabilecek eleman sayısına ve yaprakta yer olup olmamasına göre değişir.
  - Yaprağa sığabilecek eleman sayısının m olduğu bir ağaç için arama ve ekleme maliyeti $O(\log(n/m))$'dir.
  - Yaprakta yer yoksa, yeni yaprak oluşturmak için bir ve son düğümü düzenlemek için bir yeni bellek dışı erişim yapılacağından toplam maliyet $O(2 + \log(n/m))$ olur.

4.1 m Nedir?

Yapraklar (veri düğümleri, data node) tek bir kayıt tutmaz. Diskte bir sayfa (page) için ayrılan boyut ne kadarsa o kadar kayıt tutar. Sunumdaki örnek: disk sayfası 4 KB ise ve bir kayıt bunun onda biri büyüklüğündeyse bir yaprağa 10 kayıt yerleşir. Bir yaprağa yerleştirilebilecek kayıt sayısına m denir.

n kayıt m'erli yapraklara dağıldığında yaklaşık n/m yaprak olur ve ağaçta izlenen yolun uzunluğu bu yaprak sayısına göre belirlenir. Bellek dışı arama maliyetinin $O(\log(n/m))$ olmasının nedeni budur.

4.2 Neden "+2"?

Yaprak dolu değilse yeni kayıt doğrudan o yaprağa yazılır ve maliyet $O(\log(n/m))$ kalır. Yaprak doluysa yeni bir yaprak açmak gerekir. Bunun bir disk erişimi yeni yapraktan, bir disk erişimi de yeni yaprağı gösterecek düğümün güncellenmesinden gelir. Böylece $O(2+\log(n/m))$ elde edilir.

Öğrenci "buradaki 2'nin bir önemi var mı?" sorusunu kendisi sordu ve cevapladı: $\log(n/m)$ terimi pratikte çok küçük çıktığı için endüstride bu maliyetin genelde yaklaşık 2 disk erişimi sayıldığını, yani sağdaki logaritmik terimin biraz göz ardı edildiğini söyledi.

> **Ders sonrası not:** Asimptotik gösterimde $O(2 + \log(n/m)) = O(\log(n/m))$'dir; sabit 2 büyük-O içinde kaybolur. Sunumdaki "2" vurgusu asimptotik değil, pratik disk erişimi sayısı açısından anlamlıdır. Ayrıca bellek içi $O(\log n)$ ve bellek dışı $O(\log(n/m))$ değerleri ağacın dengeli olduğunu varsayar.

5. Aralık Sorgusu ve Disk Düzeni

Sunumda aralık sorgusunun maliyetinin en yakın komşuyu, yani tek bir kaydı bulmakla neredeyse aynı olduğu savunuldu. Gerekçe diskin yapısına dayanıyordu. SSD'ler değil döner diskler varsayılıyordu. Veri düğümleri diskte bir iz (track) içinde yan yana diziliyse, bulunan bir elemandan sonraki en yakın eleman hemen yanındaki veri düğümündedir.

Öğrenciye göre ağaçta en soldan en sağa doğru okunduğunda veri sıralı okunur; slaytlardaki gösterimler de (P1, P2, P3, P4 ya da A, B, C, D) bunu gösterir. Veri düğümleri diskte yan yana tutulmayı başarırsa işletim sistemi düzeyinde de bütün veri sıralı durur. Çok kötü bir senaryo yoksa, yani veri düğümleri diske dağılmamışsa, bir tanesi bulunduktan sonra yanındakiler okuna okuna gidilir.

Slayttaki formül ve disk şeması (plaka/platter, iz/track, iz sektörü, disk sektörü, küme/cluster, okuma kafası/head, kol/actuator arm) şöyleydi (ekrandan, 00:20:00):

$$\text{Transfer Time} = \frac{\text{Number of Bytes Transferred}}{\text{Number of Bytes on a Track}} \times \text{Rotation Time}$$

Öğrenci, bir sayfaya erişildikten sonra disk zaten döneceği için yanındaki sayfayı okumanın ek bir dönme gecikmesi getirmediğini, bu yüzden dönme süresinin burada sıfır kabul edilebileceğini söyledi. Vardığı sonuç: "bir aralığı yakalamak, bir değeri bulmakla aynı maliyettedir."

> **Ders sonrası not:** "k-d ağacı elemanları sıralı tutmayı garanti eder" iddiası çok boyutlu veri için tam doğru değildir; birden fazla boyutta tek bir doğal sıralama yoktur. Yaprak sırası noktaları uzaysal olarak gruplar, ama bir dikdörtgen aralık birden fazla dala yayılabilir ve sorgu birçok yaprağa inmek zorunda kalabilir. Dengeli bir k-boyutlu k-d ağacında aralık sorgusunun en kötü durum maliyeti büyük olasılıkla $O(n^{1-1/k}+r)$'dir (r: dönen kayıt sayısı). Ardışık sayfaları okumanın rastgele okumadan çok daha ucuz olduğu doğrudur; "tek eleman bulmakla aynı maliyet" ise yalnız aralığın tek bir dala ve ardışık sayfalara düştüğü durumlar için geçerlidir. Ayrıca slayttaki formül dönme gecikmesini (rotational latency) değil aktarma süresini verir; sıfır kabul edilebilecek olan, ardışık okumada tekrar oluşmayan gecikmedir.

6. Prototip Uygulama: Türkiye İl Merkezleri Üzerinde k-d Ağacı

6.1 Veri Seti

Prototip, literatürdeki ilk çalışmanın bir benzeri olarak tasarlandı. O çalışma dünya üzerindeki koordinatların x ve y değerleriyle, yani enlem (latitude) ve boylamla (longitude) çalışıyordu. Öğrenci veri setini kendisi derledi: Türkiye'deki 81 il merkezinin enlem ve boylam değerleri ile birkaç ek bilgi. Enlem ve boylam üzerinde çalışıldığı için $k = 2$'dir, yani iki boyutlu bir ağaç kuruldu. Uygulama C ile yazıldı.

6.2 Uygulamanın İşlevleri

- iki boyutlu bir k-d ağacı oluşturmak,
- ağaca kayıt eklemek,
- kayıt aramak (en yakın komşu araması),
- aralık sorgusu yapmak,
- diske okunup yazılan sayfa boyutunu değiştirmek (Windows'ta standart 4 KB, ama örneğin 1 KB'a düşürülebilir),
- k-d ağacını, yani indeksi ve düğümleri, bu sayfa boyutuna uyacak biçimde diske yazmak ve diskten okumak.

6.3 Veri Yapıları

Slaytta düğüm ve yaprak içerikleri şöyle verildi (ekrandan, 00:23:05):

```
struct Node
{
    bool leftLeafPtr, rightLeafPtr, isLat;
    double data;
    int leftPage, rightPage;
    struct Node *left, *right, *parent;
};

struct City
{
    int plateNumber;
    char name[32];
    double area;
    int population;
    int populationDensity;
    int populationofCC;
    char region[32];
    double latitude;
    double longitude;
};
```

Düğüm şunları tutar: bölmenin bir enlem çizgisi mi boylam çizgisi mi olduğunu gösteren mantıksal değer (`isLat`), çocuğun yaprak mı düğüm mü olduğu bilgisi (`leftLeafPtr`, `rightLeafPtr`), bölme değeri (`data`; aramada sağa mı sola mı gidileceği buna göre belirlenir), sağdaki ve soldaki veri sayfalarının konumu (`leftPage`, `rightPage`) ve işaretçiler. Yaprak ise şehir kayıtlarından oluşur: plaka numarası, ad, yüzölçümü, nüfus, nüfus yoğunluğu, şehir merkezinin nüfusu, bölge, enlem ve boylam. En çok kullanılan iki alan enlem ve boylamdır.

Slayttaki ikinci madde: bir şehre ait kayıt, C derleyicisinde iç bellekteki hizalama boşlukları (padding) dahil 112 bayt yer kaplar. Bu değer x86-64 üzerinde `gcc` ile `sizeof(struct City)` hesaplanarak doğrulandı: 112 bayt.

6.4 Demo

Program başlatılınca sayfa boyutunu sorar (`Please write page size (byte)`) (ekrandan, 00:24:05). Kayıt 112 bayt olduğundan en küçük sayfa 112 bayt olabilir; 4 KB'ı aşan bir sayfa da seçilebilir. Demoda 1 KB seçildi. Bu durumda bir sayfaya

$$m = \left\lfloor \frac{1024}{112} \right\rfloor = \lfloor 9{,}14 \rfloor = 9$$

il sığar. Uygulama indeksi kurdu ve 12-13 kadar veri sayfası oluşturdu. Veri sayfalarının içeriği yazdırıldığında (aralık sorgusu) birbirine yakın illerin aynı sayfaya düştüğü görüldü (ekrandan, 00:25:10). Örneğin bir sayfada Edirne, Tekirdağ, Kırklareli, İstanbul ve Yalova; başka bir sayfada Samsun, Ordu, Rize, Artvin, Kars, Trabzon ve Ardahan vardı. Dolu sayfalarda 9 il bulunuyordu.

Program menüsünde şu seçenekler vardı: enlem ve boylamla kayıt bulma, yeni kayıt ekleme, veri sayfalarını yazdırma, indeks dosyasını kaydedip çıkma.

Ardından en yakın komşu araması gösterildi. Ordu'nun enlem ve boylamı girilip Samsun arandı. Ordu ile Samsun aynı veri sayfasında olduğu için uygulama Samsun kaydını buldu ve ayrıntılarını (şehir merkezinin nüfus yoğunluğu, yüzölçümü, bölge vb.) verdi. Amaç, başka bir noktanın koordinatıyla aramanın da doğru sayfaya ulaştığını göstermekti.

6.5 Sonuç

Öğrencinin vardığı sonuçlar: günlük hayatta verinin boyutu artmaktadır ve k-d ağacı hâlâ günceldir. Yapıya küçük eklemeler yapılarak farklı alanlara (meteoroloji, ağ vb.) uyarlanabilmektedir. Prototip ve bellek içi/dışı erişim analizi, k-d ağacının nasıl çalıştığını göstermiştir.

7. Hocanın Yorumları ve Sorular (Birinci Sunum)

Soru (hoca): Son çalışmada (akan veri kümeleme) hangi veri kullanılmış? **Cevap:** Akan büyük veri üzerinde çalışılmış; öğrenci veri setinin içeriğini incelemediğini, büyük olasılıkla sentetik olduğunu düşündüğünü söyledi.

Soru (hoca): Prototipteki enlem, boylam, nüfus gibi alanlar hazır bir veri setinden mi okundu? "Sentetik" derken rastgele mi üretildi? **Cevap:** Rastgele değil; 81 il merkezinin gerçek enlem ve boylam değerleri öğrenci tarafından derlendi. Öğrenci, toplanmış hazır bir veri olmadığı için buna "sentetik" dediğini açıkladı.

Soru (hoca): Sonuç slaytındaki "eski bir algoritma ama hâlâ güncelleniyor" ifadesiyle ne kastediliyor? **Cevap:** Algoritmanın temeli 1970'lerde önerildi ve 1990'da sunuldu; değişen temel algoritma değil, herkesin kendi alanına göre ona eklemeler yapıp zenginleştirmesidir. Öğrenci ifadenin "yeni eklemelerle güncelleniyor" diye düzeltilebileceğini söyledi.

> **Hocanın vurgusu:** Hoca k-d ağacının temel bir yapı olduğunu ve matematiksel arka planının sunumda iyi anlatıldığını söyledi. Kendisini daha çok ilgilendiren kısmın güncel kullanımlar ve k-d ağacının başka algoritmalarla nasıl birleştirildiği olduğunu belirtti.

8. Hash Fonksiyonlarının Saldırılara Karşı Karşılaştırılması

İkinci sunum İngilizce yapıldı. Başlık "Comparison of Hash Function Algorithms Against Attacks" idi (ekrandan, 00:31:30). Kayıtta konuşmanın büyük kısmı anlaşılmıyor; aşağıdaki içerik slaytlardan ve altyazının anlaşılır parçalarından kuruldu.

8.1 Giriş ve Beklenen Özellikler

Hash fonksiyonu, herhangi bir uzunluktaki veriyi sabit uzunlukta bir özete (digest) dönüştürür. Sunumda girdi boyutunun farklı uygulamalarda kullanılabilmesi için sınırsız (keyfi uzunlukta) olması gerektiği söylendi; parola saklama (storage) gibi uygulamalardan söz edildi.

Kriptografik hash fonksiyonunun sağlaması gereken güvenlik özellikleri slaytta şöyleydi (ekrandan, 00:32:20):

- **Çarpışma saldırısına direnç (resistance to collision attacks):** Saldırgan iki farklı mesaj M ve M' için aynı hash değerini, yani $H(M) = H(M')$ olacak bir çifti bulamamalıdır.
- **Ön görüntü saldırısına direnç (resistance to preimage attacks):** Ön görüntü saldırısında, verilen hash değerine giden en az bir mesajın var olduğu varsayılır. Saldırgan verilen $H(M)$ değerinden asıl veri M'yi elde edememelidir.

8.2 Standart Hash Algoritmaları

Slaytlardaki açıklamalar (ekrandan, 00:35:00 ve 00:37:00):

- **MD5:** MD ailesinin popüler bir üyesidir; Rivest tarafından 1991'de tasarlanmıştır. Merkle-Damgård yapısını kullanır ve keyfi uzunluktaki mesajdan 128 bitlik çıktı üretir. MD5'e karşı birçok saldırı bulunmuştur.
- **RIPEMD-160:** RIPEMD ailesinin bilinen üyesidir; Dobbertin, Bosselaers ve Preneel tarafından 1996'da tasarlanmıştır. O da Merkle-Damgård yapısını kullanır ve 160 bitlik özet üretir. Yarı serbest başlangıçlı (semi free-start) çarpışma, ön görüntü ve çarpışma saldırıları bulunmuştur.
- **SHA-1:** 1995'te yayımlanmıştır ve aynı yapıyı kullanır. Çok yaygın kullanıldı, ama sunumda 2011'de çok zayıfladığı için standarttan çıkarıldığı söylendi.
- **SHA-2:** NIST 2002'de SHA ailesine 256, 384 ve 512 bit hash uzunluklu SHA-256, SHA-384 ve SHA-512'yi ekledi. Bunlar MD5 ve SHA-1 ile aynı yapıyı izler, ama sıkıştırma fonksiyonuna doğrusal olmayan bir fonksiyon eklendiği için daha karmaşıktır. Bitcoin, eşler arası işlemlerin güvenliği için Hashcash'te SHA-256 kullanır. Slaytta SHA-2'nin SHA-1 kadar hızlı olmadığı için bütünlük kontrolünde tercih edilmediği ve SHA-256'nın çok iş parçacıklı (multithreading) çalışamadığı için işlemler için yeterince hızlı olmadığı da yazıyordu. Slaytın köşesinde SHA-2 sıkıştırma fonksiyonunun tur şeması vardı (A-H yazmaçları, Ch, Ma, Σ_0, Σ_1 blokları).
- **SHA-3:** Keccak algoritmasına dayanır ve sünger (sponge) yapısını kullanır.

  Ders sonrası not: Sunumda SHA-1'in standarttan çıkarılmasına NSA'nın karar verdiği söylendi. Bu kararı veren kurum büyük olasılıkla NIST'tir (2011'de kullanımdan kaldırma, sonraki yıllarda imzalar için yasaklama). Slayttaki "256, 348 and 512 bits" ifadesindeki 348 de 384 olmalıdır (SHA-384). SHA-2'nin SHA-1 kadar hızlı olmadığı doğrudur, ama SHA-1 kırıldığı için bütünlük kontrolünde de bugün SHA-2 ya da SHA-3 önerilir.

8.3 Özelliklere Göre Karşılaştırma Tablosu

Slayttaki tablo (ekrandan, 00:40:00):

Özellik	MD5	RIPEMD-160	SHA-1	SHA-2 256/512	SHA-3 256/512
Blok boyutu	512 bit	512 bit	512 bit	512/1024 bit	1088/576 bit
Kelime boyutu	32 bit	32 bit	32 bit	32/64 bit	320/320 bit
Çıktı boyutu	128 bit	160 bit	160 bit	256/512 bit	1600/1600 bit
Tur sayısı	18	80	80	64/80	24/24
İşlemler	ADD, XOR, AND, OR, NOT, SHIFT	ADD, ROTATE, XOR, AND, OR, NOT	ADD, XOR, AND, OR, NOT, ROTATE	ADD, XOR, OR, AND, SHIFT, ROTATE	-
Yapı	Merkle-Damgård	Merkle-Damgård	Merkle-Damgård	Merkle-Damgård	Sponge

Altyazıda da RIPEMD-160 ve SHA-1 için 80 tur, SHA-2 için 64/80, SHA-3 için 24 tur geçiyor; algoritmaların ADD, XOR, NOT, SHIFT gibi işlemler kullandığı söylendi.

> **Ders sonrası not:** Tablodaki birkaç değer hatalıdır. MD5'te 18 tur yoktur; büyük olasılıkla 4 tur × 16 adım = 64 adım vardır. SHA-3 sütununda 1600 bit, Keccak'ın iç durum (state) boyutudur, çıktı boyutu değildir; SHA3-256 ve SHA3-512'nin çıktısı 256 ve 512 bittir. Keccak'ta kelime (lane) boyutu 320 değil 64 bittir. SHA-3 sütunundaki işlemler boş bırakılmış; Keccak büyük olasılıkla XOR, AND, NOT ve ROTATE işlemlerini kullanır.

8.4 Saldırı Türleri

Çarpışma saldırıları ve doğum günü paradoksu. Çarpışma saldırıları genelde doğum günü paradoksu (birthday paradox) ilkesini izler: n bitlik çıktısı olan bir hash fonksiyonunda çar-

pışma bulmak için yaklaşık $2^{n/2}$ girdi denemek yeterlidir. Bu genel (jenerik) sınırdır ve her hash fonksiyonuna uygulanır.

> **Ek bilgi (derste anlatılmadı):** Doğum günü paradoksu, 23 kişilik bir grupta iki kişinin aynı gün doğmuş olma olasılığının %50'yi geçmesidir. Aynı mantıkla $N = 2^n$ olası çıktı varken yaklaşık $\sqrt{N} = 2^{n/2}$ rastgele girdiden sonra iki girdinin aynı çıktıyı vermesi beklenir. Bu yüzden 128 bitlik MD5'in jenerik çarpışma güvenliği 2^{64}, 256 bitlik SHA-256'nınki 2^{128} düzeyindedir.

Hızlı çarpışma (fast collision) saldırıları. Saldırgan hash fonksiyonunun belirli algoritmik zayıflıklarını ya da tasarım kusurlarını kullanarak çarpışmayı $2^{n/2}$'den daha az zamanda bulur. Yani çarpışmayı bulma süresi jenerik saldırıdan çok daha kısadır.

Serbest başlangıçlı (freestart) çarpışma. Sunumda freestart çarpışmanın hash fonksiyonunun kırıldığı anlamına gelmediği söylendi; yalnız bu fonksiyonların analizinde daha fazla araştırma yapılması gerektiğini gösterir. Daha fazla araştırmayla kırılma olasılığı vardır.

8.5 Saldırıların Karşılaştırması

Slayttaki ikinci tablo bilinen saldırıları ve karmaşıklıklarını veriyordu (ekrandan, 00:41:10):

Algoritma	Saldırı türü	Karmaşıklık
MD5	Çarpışma	2^{39}
MD5	Hızlı çarpışma	2^{18}
RIPEMD-160	Çarpışma	2^{67}
RIPEMD-160	Ön görüntü	$2^{158.91}$
SHA-1	Çarpışma	$< 2^{69}$
SHA-1	Çarpışma	2^{61}
SHA-1	Freestart çarpışma	-
SHA-2 (256)	Ön görüntü	$2^{255.5}$
SHA-2 (512)	Ön görüntü	$2^{511.2}$
SHA-3 (256)	Pratik çarpışma ve yakın çarpışma	-
SHA-3 (512)	İlk çarpışma olasılığı	-

Tablonun yanında kaynaklar listelenmişti; ilk sıradaki X. Wang ve H. Yu'nun "How to Break MD5 and Other Hash Functions" çalışmasıdır. Sunumda bu çalışmanın MD5'i 2^{39} karmaşıklıkla kırdığı söylendi. RIPEMD-160 için çarpışma ve ön görüntü saldırılarının karmaşıklıkları da bu tablodan anlatıldı. Kaynak listesindeki SHA-3 çalışmalarının başlıkları "New attacks on Keccak-224 and Keccak-256" ve "Collision Attacks on Up to 5 Rounds of SHA-3 Using Generalized Internal Differentials" idi; bu başlıklara göre SHA-3 satırındaki sonuçlar büyük olasılıkla azaltılmış tur sayılı (tam 24 tur değil) sürümlere aittir.

Sunumun sonucu: SHA-2 ve SHA-3 çarpışma saldırılarına karşı pek savunmasız değildir; ön görüntü saldırıları vardır, ama bunlar da çok büyük işlem gücü ve kaynak gerektirir.

9. Blokzincir İçin Algoritmalar ve Hash Fonksiyonları

Üçüncü sunumun konusu blokzincir (blockchain) için algoritmalar ve hash fonksiyonlarıydı. Akış şöyleydi: dağıtık defter, hash fonksiyonları, blokzincir mimarisi, konsensüs yapıları, dijital imza, akıllı sözleşmeler ve gelecek çalışmalar.

9.1 Dağıtık Defter

Blokzincir aslında bir dağıtık defter (distributed ledger) yapısıdır ve sürekli büyür; içine sürekli yeni işlemler eklenir. Öğrenci üç yapıyı ayırdı:

- **Merkezi yapı:** Merkez çökerse sistemin tamamı çöker. Buna tek hata noktası (single point of failure) denir. Örnek olarak banka verildi: banka herkesin parasını tutar, banka çökerse herkes etkilenir.

- **Merkeziyetsiz yapı:** Bir merkez olmadığı için bir yerin çökmesi sistemin tamamının çökmesi anlamına gelmez; sistem çalışmaya devam eder.
- **Dağıtık yapı:** Daha çok verinin kendisiyle ilgilidir: verinin bir kopyası herkeste bulunur. Herkes kimin ne kadar paraya ya da hangi veriye sahip olduğunu bilir.

Bu yapının güvenliği, herkesin kendi menfaatini koruması üzerine kuruludur: "ben kimseye yanlış yapmayayım, kimse de bana yanlış yapmasın." Bunun bir dezavantajı vardır: ağın %51'inden fazlası kötü niyetli olursa ağın kontrolü ele geçirilebilir. Ama yüzde elli artı biri ele geçirmek genelde kolay değildir.

9.2 Hash Fonksiyonları

Hash fonksiyonu bir girdiyi sabit uzunlukta bir karakter dizisine dönüştürür. Girdinin de bir üst sınırı vardır; örneğin SHA-256 en fazla $2^{64}-1$ bitlik girdi alır. Blokzincirde kullanılan farklı hash fonksiyonları vardır: Keccak, BLAKE2, SHA-3. Sunum genel olarak SHA-256 üzerinden ilerledi.

Önemli iki özellik:

- **Çarpışma direnci (collision resistance):** İki farklı girdinin aynı çıktıyı vermesi istenmez.
- **Dövülemezlik (non-malleability):** Bir x girdisinin hash'i bilindiğinde, girdide yapılan bir değişikliğin çıktıyı nasıl değiştireceği öngörülememelidir; girdi ile çıktı arasında kestirilebilir bir ilişki kurulamamalıdır.

  Ders sonrası not: Sunumda Keccak'ın "SHA-256'nın biraz değiştirilmiş bir sürümü" olduğu söylendi. Keccak, NIST'in SHA-3 yarışmasını kazanan ve SHA-3'ün temelini oluşturan algoritmadır; SHA-2 (SHA-256) ailesinden değildir ve sünger yapısını kullanır. Ethereum büyük olasılıkla standart SHA-3'ten dolgu (padding) kuralıyla ayrılan Keccak-256'yı kullanır.

9.3 Blokzincir Mimarisi

Blokzincir adından da anlaşıldığı gibi bloklardan ve zincirlerden oluşur. Her blok bir önceki bloğu işaret eder. Slayttaki blok şeması şöyleydi (ekrandan, 01:00:00):

```
Block i
  Header
    Hash(Prev)    Nonce
    Difficulty    Timestamp
    Merkle Root i
  Transactions
    T1, T2, T3...
```

- `Hash(Prev)`: önceki bloğun başlığının (header) hash değeri. Bloklar bu alanla birbirine bağlanır.
- `Difficulty`: zorluk değeri; konsensüs mekanizmasıyla ilgilidir.
- `Nonce`: Proof of Work'te aranan değer.
- `Timestamp`: bloğun oluşturulmaya başlandığı andaki zaman damgası.
- `Merkle Root`: işlemlerin Merkle ağacının kökü.
- `Transactions`: işlemler listesi. Bir işlem, örneğin bir kişinin başka birine 5 Bitcoin göndermesi gibi, iki taraf arasındaki bir aktarımdır.

9.4 Merkle Ağacı

Merkle kökü, Merkle ağacının (Merkle tree) köküdür. Merkle ağacı işlemlerin değiştirilmediğini garanti eden bir ağaç yapısıdır. Slayttaki örnek dört işlemle kuruldu (ekrandan, 00:56:30):

1. İşlemler T_1, T_2, T_3, T_4'tür. Her birinin ayrı ayrı hash'i alınır: H_1, H_2, H_3, H_4.
2. H_1 ile H_2 birleştirilip yeniden hash'lenir: H_{12}. H_3 ile H_4 birleştirilip hash'lenir: H_{34}.
3. H_{12} ile H_{34} birleştirilip hash'lenir: H_{1234}. Bu Merkle köküdür.

Ne işe yarar? T_4 değiştirilirse H_4 değişir; H_4 değişirse H_{34} değişir; H_{34} değişirse H_{1234} değişir. Yalnız Merkle kökü tutularak listedeki herhangi bir işlemin değiştirilip değiştirilmediği anlaşılabilir. Merkle kökünün blokzincirde kullanılmasının ve önemli olmasının nedeni budur.

9.5 Konsensüs Mekanizmaları ve Proof of Work

Konsensüs, fikir birliğine varma mekanizmasıdır. Blokzincirde herkesin "bu blok doğrudur, buradan devam edebiliriz" demesini sağlar ve zincire yeni bloğu kimin ekleyeceğine karar verir. Kullanılan mekanizmalar: Proof of Work, Proof of Stake, Delegated Proof of Stake. Sunumda Proof of Work anlatıldı.

i. bloğun oluşturulduğunu düşünelim. Önceki blok zaten oluşmuştur; onun başlığının hash'i alınıp `Hash(Prev)` alanına yazılır. Zorluk değeri ağ tarafından otomatik belirlenir. Zaman damgası bloğun oluşturulmaya başlandığı andır. Ağda bekleyen işlemlerden oluşan bir havuz (transaction pool) vardır. Bu işi yapan kişiye, yani Proof of Work'ü çalıştırana madenci (miner) denir. Madenci havuzdan istediği işlemleri seçebilir; tamamen kendi tercihidir. Bloğa sığabilecek en fazla işlemi aldığı varsayıldı.

Bu durumda başlıkta zorluk, Merkle kökü, zaman damgası ve `Hash(Prev)` hazırdır; eksik olan yalnız nonce'tur. Nonce birden başlayıp her denemede artan bir sayıdır. Hangi nonce'un kabul edileceği şöyle belirlenir: başlığa nonce eklenip başlığın SHA-256 hash'i alındığında hash baştan belirli sayıda sıfırla başlamalıdır.

Slaytta iki hash örneği vardı (ekrandan, 01:00:00):

```
0015f2c7411f70461fd5c0e2a943
00000dd3ba505ba0d45e99e61092
```

Üstteki hash için nonce bulmak alttakine göre daha kolaydır, çünkü baştaki sıfırların sayısı azdır. Nonce her artırıldığında tamamen rastgele görünen bir çıktı elde edilir; bu yüzden nonce'u bulmak tamamen bir deneme işidir. Doğru değer ilk denemede de çıkabilir, bir milyonuncu denemede de.

Zorluk (difficulty) bunu ayarlar. Bitcoin'de bir bloğun yaklaşık her 10 dakikada bir bulunması hedeflenir ve bu her 2016 blokta bir kontrol edilir. Bloklar 10 dakikadan hızlı bulunuyorsa istenen baştaki sıfır sayısı artırılır (sunumdaki temsilî örnekle 5'ten 10'a); böylece madencilerin bloğu bulması zorlaşır.

> **Ek bilgi (derste anlatılmadı):** Bitcoin'de zorluk aslında "baştaki sıfır sayısı" olarak değil, bir hedef (target) sayı olarak tanımlanır: başlığın hash'i bu hedeften küçük olmalıdır. Baştaki sıfır sayısı bu koşulun kaba bir göstergesidir. Ayarlama her 2016 blokta bir yapılır ve bloklar yavaş bulunuyorsa zorluk düşürülür de.

Madenciler bu işi neden yapar? Bloğu bulan kişi blok ödülü alır; sunumun yapıldığı dönemde bu ödül 6,25 Bitcoin'di. Ayrıca işlemleri kaydettiği için işlem ücreti (fee) alır; bunu bankada başka birine para gönderirken alınan ücrete benzetmek mümkündür.

> **Ders sonrası not:** Bitcoin blok ödülü yaklaşık her 210.000 blokta yarıya iner; sunumdaki 6,25 BTC 2023 için doğrudur, 2024'teki yarılanmadan sonra büyük olasılıkla 3,125 BTC olmuştur.

9.6 Dijital İmza

Dijital imza, her işlemin doğru olduğunu, yani değiştirilmediğini ve gerçekten sahibinden geldiğini kontrol etmeye yarar. Slayttaki şema iki aşamalıydı (ekrandan, 01:02:00):

- **İmzalama (kullanıcı):** İşlemin (Tx) hash'i alınır. Kullanıcı bu hash'i kendi özel anahtarıyla (private key) şifreler. İmzalanmış değer işlemle birlikte ağa gönderilir.
- **Doğrulama (madenci):** Madenci imzayı kullanıcının açık anahtarıyla (public key) çözer ve bir hash değeri elde eder. İşlemin hash'ini de kendisi hesaplar. İki hash eşitse işlem doğrudur.

  Ek bilgi (derste anlatılmadı): "Özel anahtarla şifreleme" anlatımı RSA tipi imzalar için kullanılan bir sadeleştirmedir. Bitcoin ve Ethereum büyük olasılıkla eliptik eğri tabanlı ECDSA imzası kullanır. ECDSA'da hash şifrelenip çözülmez; imza, hash ve

özel anahtardan hesaplanan bir sayı çiftidir ve açık anahtarla matematiksel bir eşitlik kontrol edilerek doğrulanır. Fikir yine aynıdır: imzayı yalnız özel anahtar sahibi üretebilir, herkes açık anahtarla doğrulayabilir.

9.7 Akıllı Sözleşmeler

Akıllı sözleşmeler (smart contracts) Ethereum ile ortaya çıkmış bir mekanizmadır. Yazılan kodun Ethereum ağı, yani blokzincir ağı üzerinde çalışması gibi düşünülebilir. Kod `if-else` gibi yapılardan oluşur. Yazılan kod Ethereum bytecode'una çevrilir ve Ethereum sanal makinesinde (EVM) çalıştırılır. Akıllı sözleşme kodunun değişip değişmediğinin kontrolünde de dijital imzalar kullanılır.

9.8 Gelecek Çalışmalar ve Çarpışma Direncinin Önemi

Sunumda sayılan gelecek çalışma yönleri:

- Proof of Work'ü daha hızlı çözebilecek saldırılara karşı daha dirençli hash fonksiyonları geliştirmek.
- Enerji tasarrufu: Proof of Work'te binlerce madenci aynı işlemi aynı anda yapmaya çalışır ve bu çok büyük bir enerji israfıdır. Sunumda Bitcoin'in toplam enerji tüketiminin Türkiye'nin toplam tüketiminden fazla olduğu söylendi.
- Blokzincirin sanal gerçeklik, artırılmış gerçeklik ve metaverse gibi yeni alanlarda kullanımı.
- Sistemin güvenliğini ve bütünlüğünü sağlamak için hash fonksiyonları sürekli saldırı altında olduğundan daha güçlü mekanizmalar geliştirmek.

Öğrenci sonda çarpışma direncinin blokzincirde neden önemli olduğuna döndü. Bloklar `Hash(Prev)` ile önceki bloğa bağlanır. Hash fonksiyonunda çarpışma bulunabilseydi, bir bloğun hash'i aynı anda başka bir bloğun hash'ine de eşit olabilirdi; yani bir blok hem önceki bloğu hem de başka bir bloğu işaret edebilirdi. Zincirin hangi bloğa bağlı olduğu belirsizleşirdi. Çarpışma direnci bu yüzden temel bir gereksinimdir.

Örnek Sorular

Soru 1 (derste sözlü, 00:17:57; ekrandan, 00:16:25): Yaprağına m kayıt sığan bir k-d ağacında yaprak doluyken yapılan eklemenin bellek dışı maliyeti $O(2 + \log(n/m))$'dir. Buradaki 2 nereden gelir ve pratikte önemi var mıdır?

> **Cevap (derste verildi):** Bir disk erişimi yeni yaprağı açmak, bir disk erişimi de yeni yaprağı gösterecek düğümü güncellemek içindir. $\log(n/m)$ terimi pratikte çok küçük kaldığı için endüstride bu maliyetin yaklaşık 2 disk erişimi sayıldığı söylendi. Asimptotik olarak ise $O(2 + \log(n/m)) = O(\log(n/m))$'dir.

Soru 2 (derste sözlü, 00:17:03 ve 00:24:20; ekrandan, 00:23:05): Bir kayıt 112 bayt yer kaplıyorsa 1 KB'lık bir veri sayfasına kaç kayıt sığar? Kayıt 4 KB'ın onda biri kadarsa 4 KB'lık sayfaya kaç kayıt sığar?

> **Cevap (derste verildi):** $m = \lfloor 1024/112 \rfloor = \lfloor 9{,}14 \rfloor = 9$; demoda dolu sayfalarda 9 il görüldü. İkinci durumda $m = 10$'dur.

Soru 3 (derste sözlü, 00:56:48; ekrandan, 00:56:30): Dört işlemli ($T_1..T_4$) bir Merkle ağacında T_4 değiştirilirse hangi hash değerleri değişir?

> **Cevap (derste verildi):** H_4, ardından H_{34}, ardından kök H_{1234} değişir. H_1, H_2, H_3, H_{12} değişmez. Kök değiştiği için yalnız Merkle kökü saklanarak değişiklik fark edilir.

Soru 4 (ekrandan, 00:59:51): Proof of Work'te istenen hash'lerden biri `0015f2c7...`, diğeri `00000dd3...` biçimindedir. Hangisi için nonce bulmak daha kolaydır, neden?

> **Cevap (derste verildi):** Baştaki sıfır sayısı az olan üstteki hash için daha kolaydır; her deneme rastgele bir çıktı verdiğinden baştaki sıfırları tutturmak deneme sayısını artırır.

Çözüm (derste verilmedi): Hash on altılık tabanda yazılmıştır ve her hane 4 bittir. Çıktı düzgün dağılmış kabul edilirse, ilk z hanenin sıfır olma olasılığı 16^{-z}'dir; beklenen deneme sayısı 16^z olur. Üstteki için $z = 2$: $16^2 = 256$ deneme. Alttaki için $z = 5$: $16^5 = 1.048.576$ deneme. Alttaki yaklaşık $16^3 = 4096$ kat zordur (python ile hesaplandı).

Dersin Toparlaması

Bu derste üç proje sunumu dinlendi. Birincisi k-d ağacını hem kuramsal hem uygulamalı yönden ele aldı: iç düğümlerin yalnız bölme bilgisini, yaprakların veriyi tuttuğu bir yapı; bellek içi $O(\log n)$, bellek dışı $O(\log(n/m))$ ve dolu yaprakta $O(2 + \log(n/m))$ maliyet; disk sayfa boyutuna göre ayarlanan bir C prototipi. Hoca k-d ağacının güncel kullanımlarının ve başka yöntemlerle birleştirilmesinin daha ilginç olduğunu vurguladı.

İkinci ve üçüncü sunumlar hash fonksiyonları üzerineydi. İkincisi standart hash algoritmalarını (MD5, RIPEMD-160, SHA-1, SHA-2, SHA-3) yapı, tur sayısı ve bilinen saldırıların karmaşıklığı açısından karşılaştırdı; jenerik çarpışma sınırı $2^{n/2}$ ile algoritmaya özgü saldırıların farkını gösterdi. Üçüncüsü hash fonksiyonlarının blokzincirde nasıl kullanıldığını anlattı: `Hash(Prev)` ile zincirleme, Merkle ağacıyla işlem bütünlüğü, nonce aramasıyla Proof of Work ve dijital imzayla işlem doğrulama.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.